device data storage and remote management of devices. Using Etherios Cloud Connector, you can easily develop cloud-based applications for connected devices that quickly scale from dozens, to hundreds or even millions of endpoints.

Prerequisites for Etherios Cloud Connector

Etherios Cloud Connector can run on any device that has a minimum of 2.5 kB of RAM and 32 kB of Flash memory. A unique feature of the Etherios Cloud Connector is that it is OS independent, which means you don't need an OS running on your device to connect to Device Cloud by Etherios.

Features

By integrating Etherios Cloud Connector into your device, you instantly enable the power of Device Cloud device management capabilities and application enablement features for your device:

- Send data to Device Cloud
- Receive data from Device Cloud
- Enable remote control of devices via the Device Cloud platform, including:
  - Firmware updates
  - Software downloads
  - Configuration edits
  - Access to file systems
  - Reboot devices

Communicating with your device

To manage your device remotely, log in to your Device Cloud account and navigate to the Device Management tab. Alternatively, you can communicate with your device programmatically by using Device Cloud Web Services.

Device Cloud Web Services requests are used to send data from a remote application (written in Java, Python, Ruby, Perl and C#) to Device Cloud, which then communicates with the device. This allows for bi-directional M2M communication.

Source code structure

The Etherios Cloud Connector source code is divided into two partitions.

- *Private partition:* The private partition includes the sources that implement the Etherios Cloud Connector public API.
- *Public Application Framework:* The Public Application Framework includes a set of sample applications used for demonstration purposes.

It also has a HTML help system plus pre-written platform files for Linux, which facilitate easy integration onto devices running any Linux OS, i.e., even a Linux PC.

You can download Etherios Cloud Connector for free from *http://www.etherios. com/products/devicecloud/connector/embedded*. Extract it and you will see the following contents:

```
connector/docs > API reference manual
connector/private ->The protocol core
connector/public -> Application framework
connector/tools -> Tools for generating configuration files
```

The threading model

Etherios Cloud Connector can be deployed in a multi-threaded or round robin control loop environment.

In multi-threaded environments that include pre-emptive threading, Etherios Cloud Connector can be implemented as a separate standalone thread by calling *connector_run()*. This is a blocking call that only returns due to a major system failure.

Alternatively, when threading is unavailable, e.g., in devices without an OS, typically in a round robin control loop or fixed state machine, Etherios Cloud Connector can be implemented using the non-blocking *connector_step()* call within the round robin control loop.

Etherios Cloud Connector execution guidelines

Here we will try to run Etherios Cloud Connector on a PC running Linux. Similarly, you can port to any device with or without an OS.

Go to */connector/public/step/platforms*

You need to create a folder for your custom platform here. If you have a Linux platform, then go to */connector/public/step/platforms/linux*

Here you can see platform specifics like:

```
os.c -> OS routines like app_os_malloc(), app_os_free(), app_
os_get_system_time(), app_os_reboot() which defines how to do
malloc and get system time on your platform.
```

For Linux, these are already defined, so open *config.c* and go to *app_get_mac_addr()*

```
#define MAC_ADDR_LENGTH 6
static uint8_t const device_mac_addr[MAC_ADDR_LENGTH] =
{0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
static connector_callback_status_t app_get_mac_
addr(connector_config_pointer_data_t * const config_mac)
{
#error "Specify device MAC address for LAN connection"
ASSERT(config_mac->bytes_required == MAC_ADDR_LENGTH);
config_mac->data = (uint8_t *)device_mac_addr;
return connector_callback_continue;
}
```

This *callback* defines how to get the MAC address of your device, and for testing you may hardcode MAC and rewrite it as follows:

```
#define MAC_ADDR_LENGTH 6
```